1. What does the recursive acronym PHP stand for?

(A) PHP: Hypertext Preprocessor
(B) Personal Home Page
(C) Private Hosting Protocol
(D) Programmed Hypertext Processor

2. Server-side PHP code execution occurs on the:

(A) Web Server before sending pure HTML output to the client browser
(B) Client browser using a JavaScript engine
(C) Operating system kernel directly
(D) Database server only

3. Which tags are standardly used to delimit PHP code within a server-side script?

(A) <?php ... ?>
(B) <% ... %>
(C) <script php> ... </script>
(D) <? ... ?>

4. In PHP syntax, every variable name must begin with which mandatory prefix character?

(A) $ (dollar sign)
(B) # (hash)
(C) @ (at symbol)
(D) % (percent)

5. Which PHP language construct outputs one or more strings to the response output buffer?

(A) echo (or print)
(B) output()
(C) display()
(D) write()

6. In PHP, which string concatenation operator is used to join two strings together?

(A) . (dot operator)
(B) + (plus operator)
(C) & (ampersand)
(D) ~ (tilde)

7. What statement terminates every standard executable statement in PHP?

(A) ; (semicolon)
(B) : (colon)
(C) . (dot)
(D) Newline

8. Which PHP comment syntax is valid for single-line comments?

(A) // or #
(B) /* only */
(C) <!-- -->
(D) -- only

9. Which PHP data type represents a collection of key-value pairs or indexed elements?

(A) Array
(B) Dictionary
(C) List
(D) Collection

10. In PHP, arrays that use named string keys rather than numeric indices are called:

(A) Associative Arrays
(B) Indexed Arrays
(C) Multidimensional Arrays
(D) Hash Vectors

11. Which built-in PHP superglobal array contains variables passed to the current script via the URL query string?

(A) $_GET
(B) $_POST
(C) $_REQUEST
(D) $_SERVER

12. Which PHP superglobal array contains form data sent to the script via HTTP POST?

(A) $_POST
(B) $_GET
(C) $_FILES
(D) $_ENV

13. Which PHP superglobal contains uploaded file metadata (such as name, size, tmp_name, and error)?

(A) $_FILES
(B) $_POST
(C) $_UPLOAD
(D) $_INPUT

14. Which PHP superglobal array contains server and execution environment information (e.g., HTTP_HOST, REMOTE_ADDR)?

(A) $_SERVER
(B) $_ENV
(C) $_GLOBAL
(D) $_SYSTEM

15. Which PHP function is used to create or send an HTTP cookie to the client browser before any output is sent?

(A) setcookie(name, value, expire, path)
(B) createCookie()
(C) cookie_set()
(D) $_COOKIE['name'] = val

16. Which PHP function must be called at the very beginning of a script to initialize or resume a user session?

(A) session_start()
(B) session_init()
(C) start_session()
(D) session_resume()

17. Which superglobal array stores session variables across multiple page requests for an authenticated user?

(A) $_SESSION
(B) $_COOKIE
(C) $_SERVER
(D) $_GLOBAL

18. Which PHP function completely frees all session variables and destroys the session data on the server?

(A) session_destroy()
(B) session_unset()
(C) session_kill()
(D) session_clear()

19. Which modern database abstraction library in PHP provides a consistent interface to connect to MySQL, PostgreSQL, and SQLite?

(A) PHP Data Objects (PDO)
(B) MySQLi
(C) mysql_connect
(D) ODBC Native

20. What does MySQL stand for in relational database management systems?

(A) My Structured Query Language
(B) Multi-Yield SQL
(C) Modular Query Language
(D) Mainframe SQL

21. Which SQL statement is used to retrieve data records from a relational database table?

(A) SELECT
(B) GET
(C) RETRIEVE
(D) PULL

22. Which SQL statement adds new rows of data into a database table?

(A) INSERT INTO
(B) ADD ROW
(C) APPEND
(D) CREATE DATA

23. Which SQL statement modifies existing records within a database table?

(A) UPDATE
(B) MODIFY
(C) ALTER
(D) CHANGE

24. Which SQL statement removes existing records from a database table matching a WHERE clause?

(A) DELETE FROM
(B) REMOVE
(C) DROP
(D) TRUNCATE WHERE

25. What does XML stand for in web data exchange?

(A) eXtensible Markup Language
(B) Extended Machine Language
(C) External Modern Language
(D) Executable Markup Level

26. Unlike HTML, which has predefined presentation tags, XML is fundamentally designed to:

(A) Store and transport structured data using custom user-defined tags
(B) Render styled graphics on screen
(C) Style web pages with colors
(D) Execute server scripts

27. An XML document is said to be 'Well-Formed' if it:

(A) Adheres strictly to XML syntax rules (single root element, matching closed tags, quoted attributes, case sensitivity)
(B) Has a CSS stylesheet attached
(C) Is stored in a MySQL database
(D) Contains fewer than 100 lines

28. An XML document is said to be 'Valid' if it:

(A) Is well-formed AND conforms to a formal Document Type Definition (DTD) or XML Schema (XSD)
(B) Opens without errors in Notepad
(C) Is digitally signed with RSA
(D) Can be converted to JSON

29. What is a DTD in XML technologies?

(A) Document Type Definition (defines legal building blocks, elements, and attributes of an XML document)
(B) Data Transfer Document
(C) Digital Text Descriptor
(D) Dynamic Tag Directory

30. What does XSD stand for in modern XML data validation?

(A) XML Schema Definition
(B) XML Style Document
(C) XML System Data
(D) XML Structure Definition

31. Why is XML Schema (XSD) generally preferred over legacy DTD for validating enterprise XML?

(A) XSD is written in XML syntax itself and supports rich data typing (integers, dates, booleans), whereas DTD lacks data types
(B) XSD runs faster on CPU
(C) DTD is not supported by Java
(D) XSD allows unclosed tags

32. Which XML query language is used to navigate through elements and attributes in an XML document tree?

(A) XPath (XML Path Language)
(B) XQuery only
(C) XPointer
(D) SQL-XML

33. Which language is used to transform XML documents into other formats, such as HTML, XHTML, or plain text?

(A) XSLT (Extensible Stylesheet Language Transformations)
(B) CSS
(C) Sass
(D) DOM-XML

34. What does REST stand for in web service architectures?

(A) Representational State Transfer
(B) Remote Execution System Tool
(C) Reliable Endpoint State Transport
(D) Request Entity Standard Type

35. In RESTful API design, resources are identified using:

(A) Uniform Resource Identifiers (URIs / URLs)
(B) Method names in query strings
(C) Operating system file paths
(D) Database row pointers

36. Which HTTP method is standardly mapped to creating a new resource in a RESTful API?

(A) POST
(B) GET
(C) PUT
(D) DELETE

37. Which HTTP method is standardly mapped to reading or retrieving an existing resource in REST?

(A) GET
(B) POST
(C) PUT
(D) HEAD

38. Which HTTP method is standardly mapped to completely updating or replacing an existing resource in REST?

(A) PUT
(B) POST
(C) PATCH
(D) GET

39. Which HTTP method is mapped to applying partial modifications to a resource in REST?

(A) PATCH
(B) PUT
(C) POST
(D) UPDATE

40. Which HTTP method is mapped to removing or deleting a resource in REST?

(A) DELETE
(B) REMOVE
(C) DROP
(D) KILL

41. What is the primary architectural difference between SOAP and REST web services?

(A) SOAP is a strict, XML-based protocol with formal WSDL contracts; REST is an architectural style leveraging standard HTTP verbs and flexible payloads (JSON/XML)
(B) SOAP uses JSON; REST uses XML
(C) SOAP cannot run over HTTPS
(D) REST requires heavyweight middleware

42. What is WSDL in SOAP web service architectures?

(A) Web Services Description Language (an XML grammar describing network services, operations, and message formats)
(B) Web Socket Definition Level
(C) Web Security Data Layer
(D) Wide Service Directory Link

43. What are the four components of a SOAP message envelope?

(A) Envelope, Header (optional), Body (mandatory payload), and Fault (error handling)
(B) Root, Head, Content, Tail
(C) Start, Message, Signature, End
(D) Request, Response, Status, Code

44. What is SQL Injection (SQLi) in database-backed web applications?

(A) An attack where malicious SQL statements are inserted into entry fields to manipulate backend database queries
(B) Overloading database RAM with queries
(C) Corrupting table indexes via disk failure
(D) Intercepting database passwords on Wi-Fi

45. What is the primary, infallible defense against SQL Injection in PHP web applications?

(A) Using PDO or MySQLi Prepared Statements with Parameterized Queries (separating SQL structure from user data)
(B) Escaping strings with addslashes()
(C) Removing all single quotes with regex
(D) Using base64 encoding on inputs

46. Why are functions like addslashes() or raw string concatenation vulnerable to SQL injection bypasses?

(A) Character encoding manipulations (e.g. multibyte GBK character attacks) and numeric field injections bypass quote escaping completely
(B) They slow down server performance
(C) They corrupt UTF-8 text
(D) They only work on Windows servers

47. How do Prepared Statements guarantee absolute immunity against SQL injection?

(A) The database compiles the SQL query template and execution plan first; user parameters are sent separately as pure data, never parsed as executable SQL commands
(B) By encrypting database tables with AES
(C) By hashing query text with SHA-256
(D) By converting queries to stored procedures only

48. What is Cross-Site Scripting (XSS) in server-side web development?

(A) An attack where an attacker injects malicious client-side JavaScript that is reflected by the server or stored in the database and executed in victims' browsers
(B) Executing PHP code on the client
(C) Stealing server hard drives
(D) Flooding the server with HTTP GET requests

49. Which built-in PHP function is standardly used to neutralize XSS by converting special characters (like <, >, &, ") into HTML entities?

(A) htmlspecialchars($input, ENT_QUOTES, 'UTF-8')
(B) strip_tags()
(C) urlencode()
(D) html_entity_decode()

50. What is Cross-Site Request Forgery (CSRF)?

(A) An attack that forces an authenticated victim's browser to send forged HTTP requests (with ambient session cookies) to a vulnerable application
(B) Stealing user passwords via keylogger
(C) Injecting SQL into form fields
(D) Intercepting SSL packets

51. How does the Anti-CSRF Synchronizer Token Pattern defend against CSRF attacks?

(A) The server generates a cryptographically random, unpredictable token stored in the session and embedded as a hidden form field; POST submissions without a valid token are rejected
(B) By disabling cookies completely
(C) By IP-whitelisting all users
(D) By requiring CAPTCHA on every click

52. What does the 'SameSite' cookie attribute accomplish in modern browser cookie security?

(A) Restricts cookie transmission on cross-site requests: 'Strict' blocks cookies entirely cross-site; 'Lax' permits on top-level GET navigations
(B) Encrypts cookies with RSA
(C) Sets cookie expiration to 1 hour
(D) Deletes cookies on tab close

53. What does the 'HttpOnly' flag on a session cookie prevent?

(A) Prevents client-side JavaScript (`document.cookie`) from accessing the cookie, mitigating cookie theft via XSS attacks
(B) Forces cookie transmission over HTTPS
(C) Hides cookie from web server
(D) Prevents cookie deletion

54. What does the 'Secure' flag on a session cookie enforce?

(A) Instructs browsers that the cookie must strictly only be transmitted over encrypted HTTPS connections
(B) Enforces a 32-character minimum password
(C) Validates client IP addresses
(D) Encrypts cookie content on client disk

55. What is 'Session Fixation' in web authentication, and how is it prevented in PHP?

(A) An attacker forces a known session ID onto a victim before login; prevented by calling `session_regenerate_id(true)` immediately upon successful user authentication
(B) Attacker guesses session passwords; prevented by captcha
(C) Session timeout expires; prevented by keepalive
(D) Database runs out of session slots

56. What is 'Session Hijacking' and what defensive measures reduce its likelihood?

(A) An attacker sniffs or steals a valid session ID to impersonate the victim; defended using HTTPS, HttpOnly cookies, IP/User-Agent fingerprint validation, and short idle timeouts
(B) Attacker reboots the server; defended by UPS
(C) Attacker deletes user accounts; defended by backups
(D) Attacker changes CSS stylesheets

57. Which password hashing algorithm is standardly recommended and built into PHP via `password_hash()`?

(A) Bcrypt (PASSWORD_BCRYPT) or Argon2 (PASSWORD_ARGON2ID)
(B) MD5 (md5($password))
(C) SHA-1 (sha1($password))
(D) Base64 (base64_encode())

58. Why should legacy hash algorithms like MD5 and SHA-1 NEVER be used for storing passwords?

(A) They are designed to be fast, allowing modern GPUs to calculate billions of hashes per second to crack passwords via brute force and rainbow tables
(B) They only hash numbers
(C) They require 100MB of RAM
(D) They are proprietary algorithms

59. What does the 'cost' factor (work factor) parameter in `password_hash($pass, PASSWORD_BCRYPT, ['cost' => 12])` do?

(A) Increases the algorithmic computation time exponentially (2^cost iterations), slowing down brute-force attacker hardware
(B) Increases the character length of output hash
(C) Charges a monetary licensing fee
(D) Sets password expiration time

60. Which PHP function is used to securely verify that a plaintext password matches a stored password hash?

(A) password_verify($password, $hash)
(B) password_check()
(C) password_match()
(D) hash_equals(md5($p), $h)

61. What are the three parts of a JSON Web Token (JWT) separated by dots (.)?

(A) Header, Payload, and Signature
(B) Prefix, Data, and Key
(C) User, Claims, and Encryption
(D) Token, Expiry, and Digest

62. How is each component of a JSON Web Token (JWT) encoded over the wire?

(A) Base64URL encoding
(B) Hexadecimal encoding
(C) Raw ASCII text
(D) Gzip compression

63. Is the payload data inside a standard JWT encrypted and secret by default?

(A) No: it is only Base64URL-encoded (readable by anyone); the signature only guarantees authenticity and tamper-proofing (integrity)
(B) Yes: it is encrypted with 256-bit AES
(C) Yes: only the private key holder can view payload
(D) No: it is plain binary

64. In PHP, what is the difference between `require` and `include` statements when including external files?

(A) `require` halts script execution with a fatal error (E_COMPILE_ERROR) if the file is missing; `include` emits a warning (E_WARNING) and continues executing
(B) `include` stops execution; `require` continues
(C) `require` only works on PHP files
(D) They are identical aliases

65. What do `require_once` and `include_once` prevent when building modular applications?

(A) Prevents including the same file multiple times, avoiding fatal function or class re-declaration errors
(B) Caches files in RAM
(C) Encrypts file contents
(D) Minifies included scripts

66. What is PHP PDO prepared statement parameter binding via `bindParam()` vs `bindValue()`?

(A) `bindParam()` binds a variable by reference (evaluated at execute time); `bindValue()` binds a concrete value immediately
(B) `bindValue()` binds by reference
(C) There is zero functional difference
(D) `bindParam()` only accepts integers

67. In database operations, what is a Transaction, and which four properties (ACID) guarantee its reliability?

(A) A unit of work performed against a database satisfying: Atomicity, Consistency, Isolation, and Durability
(B) A SQL query; Accuracy, Completeness, Integrity, Dependability
(C) A table join; All, Create, Index, Drop
(D) A network handshake

68. How do you manage a database transaction in PHP PDO?

(A) $pdo->beginTransaction(); ... $pdo->commit(); (or $pdo->rollBack() on failure)
(B) $pdo->start(); $pdo->stop();
(C) $pdo->open(); $pdo->close();
(D) $pdo->save(); $pdo->undo();

69. What is an XML CDATA section (`<![CDATA[ ... ]]>`) used for?

(A) Enclosing blocks of text that may contain characters like < and & without having the XML parser interpret them as markup tags
(B) Encrypting XML payloads
(C) Storing CSS stylesheets in XML
(D) Declaring XML schema namespaces

70. What is XML External Entity (XXE) injection?

(A) A vulnerability where an attacker manipulates an XML parser via crafted DTD entities to disclose local server files or conduct SSRF attacks
(B) Injecting JavaScript into HTML forms
(C) Overloading XML parser memory
(D) Corrupting XML tag names

71. How is XML External Entity (XXE) injection prevented in PHP's libxml parsers?

(A) By explicitly disabling external entity loading: `libxml_disable_entity_loader(true);`
(B) By converting XML to JSON
(C) By renaming XML files
(D) By using DTD validation

72. What is the role of the PHP `filter_var()` function with filter constants (e.g., FILTER_VALIDATE_EMAIL)?

(A) Validates and sanitizes data against standard criteria (emails, URLs, IPs, integers)
(B) Encrypts strings with AES
(C) Converts strings to uppercase
(D) Filters database table rows

73. What does the HTTP header `Content-Security-Policy: default-src 'self'` enforce on clients?

(A) Restricts the browser from loading scripts, styles, or images from any external domain other than the site's own origin
(B) Enforces HTTPS encryption only
(C) Disables browser cookies
(D) Blocks all search engine crawlers

74. What does the HTTP header `X-Content-Type-Options: nosniff` prevent?

(A) Prevents browsers from MIME-sniffing a response away from the declared Content-Type, mitigating drive-by script injection
(B) Stops file uploads
(C) Forces gzip compression
(D) Hides HTTP status codes

75. What is Server-Side Request Forgery (SSRF)?

(A) An attack where an attacker induces a server-side application to make HTTP requests to an arbitrary internal or external domain (e.g., accessing cloud metadata 169.254.169.254)
(B) Attacker forges client cookies
(C) Attacker injects SQL into forms
(D) Attacker crashes client browsers

76. What is the architectural distinction between PHP Object-Relational Mappers (ORMs like Doctrine / Eloquent) and raw PDO queries?

(A) ORMs map database tables to object-oriented classes and rows to entity objects, abstracting SQL into an object API; PDO executes explicit SQL strings directly
(B) ORMs compile PHP to C++
(C) PDO cannot connect to MySQL
(D) ORMs eliminate the need for databases

77. In RESTful API design, what does the property of 'Idempotence' mean, and which HTTP methods are idempotent?

(A) An operation can be applied multiple times without changing the result beyond the initial application; GET, PUT, DELETE, and HEAD are idempotent (POST is NOT)
(B) An operation is encrypted with RSA
(C) An operation returns JSON format
(D) An operation runs in 1 millisecond

78. What is the Richardson Maturity Model for REST APIs, and what characterizes Level 3 (HATEOAS)?

(A) Level 3 incorporates Hypermedia as the Engine of Application State (HATEOAS), providing dynamic hypermedia link relations inside responses to guide client navigation
(B) Level 3 uses XML only
(C) Level 3 requires SOAP envelopes
(D) Level 3 eliminates HTTP headers

79. How does Timing Attack vulnerability occur during password or token verification, and what PHP function prevents it?

(A) Using standard `===` compares strings character-by-character and exits early upon first mismatch, leaking string length and characters via microsecond timing differences; prevented using `hash_equals()`
(B) Server clock drifting; prevented by NTP
(C) Database lag; prevented by caching
(D) Network packet delay; prevented by TCP

80. Why is `unserialize()` considered extremely dangerous when applied to untrusted user input in PHP (Insecure Deserialization)?

(A) It can instantiate arbitrary classes present in the codebase and trigger magic methods (`__destruct`, `__wakeup`), leading to Remote Code Execution (POP chains / gadget chains)
(B) It corrupts the PHP interpreter
(C) It leaks database passwords in plain text
(D) It causes infinite loop recursion

81. What should be used instead of `unserialize()` to safely deserialize data received from untrusted clients?

(A) JSON format via `json_decode()`
(B) `eval()`
(C) `base64_decode()`
(D) `parse_str()`

82. In PHP database operations, what is Connection Pooling, and why is persistent connection handling (`PDO::ATTR_PERSISTENT => true`) used carefully?

(A) Persistent connections remain open across script requests avoiding repeated TCP/MySQL handshakes; requires caution to avoid exhausting MySQL max_connections limit
(B) It encrypts MySQL traffic
(C) It merges multiple tables into one
(D) It bypasses query execution plans

83. What is the difference between Optimistic Concurrency Control and Pessimistic Concurrency Control in database-backed web applications?

(A) Optimistic uses version numbers to verify records have not changed before committing; Pessimistic acquires explicit row/table locks (SELECT ... FOR UPDATE) preventing concurrent reads/writes
(B) Optimistic locks tables; Pessimistic does not
(C) Pessimistic only works in NoSQL
(D) They behave identically

84. In XML processing, what is the difference between DOM parsers and SAX (Simple API for XML) parsers?

(A) DOM parser loads the entire XML tree into RAM (high memory for large files); SAX is a stream-based, event-driven parser that processes nodes sequentially with minimal memory
(B) SAX loads into RAM; DOM streams
(C) DOM parser cannot read attributes
(D) SAX is deprecated in HTML5

85. What is the purpose of XML Namespaces (`xmlns:prefix="URI"`) in compound XML documents?

- **(A)** Prevents element naming collisions when combining XML vocabularies from distinct schema domains (e.g., differentiating `<book:title>` from `<person:title>`)
- **(B)** Enforces DNS domain validation
- **(C)** Encrypts XML tags
- **(D)** Formats XML for printing

86. In OAuth 2.0 authorization framework (RFC 6749), what is the difference between the Authorization Code Flow and Client Credentials Flow?

- **(A)** Authorization Code flow involves end-user consent and redirects to issue user-delegated tokens; Client Credentials flow is machine-to-machine authentication without user intervention
- **(B)** Authorization code is for mobile only
- **(C)** Client credentials requires SMS
- **(D)** Authorization code uses SOAP

87. What is the PKCE (Proof Key for Code Exchange) extension in OAuth 2.0, and why is it mandatory for Single Page Apps (SPAs) and mobile apps?

- **(A)** Prevents authorization code interception attacks on public clients that cannot securely store a client secret, using dynamic cryptographic code verifiers and challenges
- **(B)** Generates SSL certificates
- **(C)** Encrypts MySQL databases
- **(D)** Validates email addresses

88. How does the 'Double Submit Cookie' pattern defend against CSRF attacks in stateless RESTful APIs?

- **(A)** The client generates a random pseudorandom token, stores it in a cookie, and submits the exact same token in an HTTP custom request header; the server verifies both match
- **(B)** Cookies are transmitted twice via UDP
- **(C)** Passwords are encrypted twice
- **(D)** Database stores two sessions per user

89. What is HTTP Request Smuggling, and how does it arise in front-end reverse proxy / backend server architectures?

- **(A)** Occurs when front-end proxies and back-end servers disagree on message boundaries due to conflicting `Content-Length` and `Transfer-Encoding: chunked` headers
- **(B)** Injecting JavaScript into GET URLs
- **(C)** Sniffing packets on open Wi-Fi
- **(D)** Overloading servers with SYN packets

90. What is Clickjacking, and what modern HTTP response headers completely eliminate this threat?

- **(A)** Tricking a user into clicking transparent overlays; prevented using `Content-Security-Policy: frame-ancestors 'none'` and `X-Frame-Options: DENY`
- **(B)** Using CAPTCHA
- **(C)** Disabling JavaScript
- **(D)** Using CORS headers

91. In PHP session security, why must session storage be moved from local filesystem (`/tmp`) to Redis or Memcached in scaled production clusters?

- **(A)** Local disk sessions cannot be shared across multiple horizontal load-balanced web server nodes; centralized Redis caching provides unified shared session access
- **(B)** Local disk cannot store strings
- **(C)** Redis encrypts sessions with AES
- **(D)** MySQL cannot read files

92. What is OpenID Connect (OIDC), and how does it extend OAuth 2.0?

- **(A)** An identity authentication layer built directly on top of OAuth 2.0 that introduces ID Tokens (JWT) to standardize user authentication alongside authorization
- **(B)** A replacement for HTTPS
- **(C)** A tool to compress XML files
- **(D)** A database query language

93. What security vulnerability arises from 'Path Traversal' (Directory Traversal) in file-download scripts (e.g., `include($_GET['page']);`)?

- **(A)** Attackers input characters like `../../etc/passwd` to navigate outside the web root and read or execute arbitrary operating system files; defended using `basename()` and whitelisting
- **(B)** Server hard drive runs out of space
- **(C)** Database tables are dropped
- **(D)** RAM memory leaks

94. What is Command Injection in server-side web scripting (e.g., passing unescaped input to `exec()` or `system()`)?

- **(A)** Attackers concatenate shell metacharacters (`;`, `&&`, `|`) to execute arbitrary operating system terminal commands on the host server; defended using `escapeshellarg()` or avoiding shell execution
- **(B)** Injecting SQL into databases
- **(C)** Injecting HTML into DOM
- **(D)** Compiling C++ scripts

95. What is Rate Limiting in Web API engineering, and which standard algorithms are used to implement it (e.g., Token Bucket, Leaky Bucket)?

- **(A)** Restricting the number of API requests a client can submit within a time window to prevent brute-force attacks and denial-of-service, utilizing Token Bucket or Sliding Window algorithms
- **(B)** Limiting broadband download speed
- **(C)** Throttling CPU clock frequency
- **(D)** Restricting database row counts

96. What are the standard HTTP response headers returned to API clients to convey rate-limiting state?

- **(A)** `X-RateLimit-Limit`, `X-RateLimit-Remaining`, and `X-RateLimit-Reset` (along with HTTP 429 Too Many Requests)
- **(B)** `Content-Limit`, `Content-Remaining`
- **(C)** `Status-Throttle`, `Status-Wait`
- **(D)** `API-Quota`, `API-Time`

97. What is the Difference between Symmetric-Key JWT Signing (HS256) and Asymmetric-Key JWT Signing (RS256)?

- **(A)** HS256 uses a single shared secret key to both sign and verify tokens; RS256 uses a private key to sign tokens and a public key to verify them across decoupled microservices
- **(B)** HS256 is 1000 bits; RS256 is 128 bits
- **(C)** RS256 only works on Linux
- **(D)** HS256 cannot be validated in JavaScript

98. What is GraphQL, and how does it solve Over-Fetching and Under-Fetching compared to traditional REST APIs?

- **(A)** A declarative query language for APIs allowing clients to request the exact data fields needed in a single roundtrip, eliminating wasted bandwidth and multi-request waterfall latency
- **(B)** A database engine like MySQL
- **(C)** A replacement for JSON
- **(D)** A network routing protocol

99. What security vulnerability is unique to GraphQL APIs, and what defensive controls must be enforced?

(A) Complex recursive nested queries causing Denial-of-Service / CPU exhaustion; defended using Query Depth Limiting, Query Cost Analysis, and disabling Introspection in production

(B) SQL injection on JSON fields

(C) Loss of SSL encryption

(D) Memory leaks in CSS

100. What is Web Application Firewall (WAF) rule tuning, and how does ModSecurity inspect HTTP payloads on Apache/Nginx servers?

(A) ModSecurity analyzes incoming request headers, query strings, and POST bodies in real time against the OWASP Core Rule Set (CRS) to detect and block SQLi, XSS, and exploit probes before reaching PHP

(B) WAF encrypts PHP scripts with Zend Guard

(C) WAF speeds up PHP execution via JIT

(D) WAF replaces database indexes

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain Server-Side Web Architecture, contrasting server-side scripting (PHP) with client-side scripting (JavaScript) and detailing the HTTP lifecycle.

Q2. Explain PHP Superglobal Arrays, detailing the purpose and security handling of $_GET, $_POST, $_SERVER, $_SESSION, $_COOKIE, and $_FILES.

Q3. Detail the implementation of secure User Authentication in PHP: password hashing with Bcrypt/Argon2, password_hash(), and password_verify().

Q4. Explain PHP Session Management mechanics, detailing session_start(), cookie parameters, session fixation, and secure session destruction.

Q5. Analyze SQL Injection (SQLi) attacks, demonstrating vulnerable code vs immune PDO Prepared Statements with Parameterized Queries.

Q6. Explain Cross-Site Scripting (XSS) prevention in server-rendered templates, detailing htmlspecialchars(), Content Security Policy, and context-aware escaping.

Q7. Explain Cross-Site Request Forgery (CSRF) defense using Anti-CSRF Synchronizer Tokens and SameSite cookies.

Q8. Explain the architecture of XML technologies: XML syntax rules, Document Type Definitions (DTD), and XML Schema (XSD).

Q9. Explain XPath and XSLT with comprehensive code examples demonstrating XML querying and HTML report transformation.

Q10. Compare SOAP and RESTful Web Services across protocol layers, data formats, statefulness, performance, and tooling.

Q11. Explain RESTful API Architectural Constraints: Statelessness, Client-Server, Cacheability, Uniform Interface, and Layered System.

Q12. Explain JSON Web Tokens (JWT) architecture, detailing Header, Payload (Claims), Signature generation, and verification workflow.

Q13. Explain PHP File Upload handling, detailing $_FILES array, validation rules (MIME, size, extension), and secure storage.

Q14. Detail PHP Data Objects (PDO) advanced error modes, transaction management, and connection configuration.

Q15. Explain Server-Side Request Forgery (SSRF) vulnerabilities, impact on cloud infrastructure, and architectural defenses.

Q16. Explain XML External Entity (XXE) injection vulnerabilities and detail how to configure XML parsers to disable entity resolution.

Q17. Discuss HTTP Security Headers: Content-Security-Policy (CSP), Strict-Transport-Security (HSTS), X-Content-Type-Options, and X-Frame-Options.

Q18. Explain the OAuth 2.0 Authorization Code Grant flow with Proof Key for Code Exchange (PKCE).

Q19. Explain Rate Limiting architectures using Redis and the Sliding Window Log / Token Bucket algorithms.

Q20. Describe Database Indexing principles (B-Tree, Hash, Composite Indexes) and explain how EXPLAIN queries identify table scan bottlenecks.

Q21. Write a complete PHP RESTful API endpoint controller implementing CRUD operations (GET, POST, PUT, DELETE) returning JSON responses.

Q22. Write a PHP class to generate and verify JSON Web Tokens (JWT) using HMAC-SHA256 without external third-party libraries.

Q23. Write a complete XML Schema (XSD) document defining a validated Student Record with ID, name, email, and age constraints.

Q24. Write a PHP script to parse an XML document using DOMDocument and XPath, querying books priced over $50.

Q25. Demonstrate how to prevent SQL Injection in dynamic search filters with optional criteria in PHP PDO.

Q26. Explain Database Connection Pooling and persistent connections in PHP, detailing why FastCGI and PHP-FPM architectures require connection sizing.

Q27. Write a PHP script to transform an XML document into an HTML table using XSLTProcessor.

Q28. Explain Two-Factor Authentication (2FA) implementation in web applications using Time-Based One-Time Password (TOTP) algorithm (RFC 6238).

Q29. Explain Content Security Policy (CSP) Reporting (report-uri / report-to) and demonstrate setting up an automated violation ingestion endpoint.

Q30. Analyze Web Application security auditing tools: OWASP ZAP, Burp Suite, SQLMap, and Nikto in penetration testing workflows.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[A]	**Q21:**	[A]	**Q41:**	[A]	**Q61:**	[A]	**Q81:**	[A]
Q2:	[A]	**Q22:**	[A]	**Q42:**	[A]	**Q62:**	[A]	**Q82:**	[A]
Q3:	[A]	**Q23:**	[A]	**Q43:**	[A]	**Q63:**	[A]	**Q83:**	[A]
Q4:	[A]	**Q24:**	[A]	**Q44:**	[A]	**Q64:**	[A]	**Q84:**	[A]
Q5:	[A]	**Q25:**	[A]	**Q45:**	[A]	**Q65:**	[A]	**Q85:**	[A]
Q6:	[A]	**Q26:**	[A]	**Q46:**	[A]	**Q66:**	[A]	**Q86:**	[A]
Q7:	[A]	**Q27:**	[A]	**Q47:**	[A]	**Q67:**	[A]	**Q87:**	[A]
Q8:	[A]	**Q28:**	[A]	**Q48:**	[A]	**Q68:**	[A]	**Q88:**	[A]
Q9:	[A]	**Q29:**	[A]	**Q49:**	[A]	**Q69:**	[A]	**Q89:**	[A]
Q10:	[A]	**Q30:**	[A]	**Q50:**	[A]	**Q70:**	[A]	**Q90:**	[A]
Q11:	[A]	**Q31:**	[A]	**Q51:**	[A]	**Q71:**	[A]	**Q91:**	[A]
Q12:	[A]	**Q32:**	[A]	**Q52:**	[A]	**Q72:**	[A]	**Q92:**	[A]
Q13:	[A]	**Q33:**	[A]	**Q53:**	[A]	**Q73:**	[A]	**Q93:**	[A]
Q14:	[A]	**Q34:**	[A]	**Q54:**	[A]	**Q74:**	[A]	**Q94:**	[A]
Q15:	[A]	**Q35:**	[A]	**Q55:**	[A]	**Q75:**	[A]	**Q95:**	[A]
Q16:	[A]	**Q36:**	[A]	**Q56:**	[A]	**Q76:**	[A]	**Q96:**	[A]
Q17:	[A]	**Q37:**	[A]	**Q57:**	[A]	**Q77:**	[A]	**Q97:**	[A]
Q18:	[A]	**Q38:**	[A]	**Q58:**	[A]	**Q78:**	[A]	**Q98:**	[A]
Q19:	[A]	**Q39:**	[A]	**Q59:**	[A]	**Q79:**	[A]	**Q99:**	[A]
Q20:	[A]	**Q40:**	[A]	**Q60:**	[A]	**Q80:**	[A]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

[Problem Statement]: Detail the implementation of secure User Authentication in PHP: password hashing with Bcrypt/Argon2, password_hash(), a
[Detailed Step-by-Step Answer]:
Storing passwords securely requires one-way cryptographic hashing with adaptive work factors:

Password Storage Architecture:
1. Registration (Hashing):
```php
$password = $_POST['password'];
// Automatically generates a cryptographically secure random salt and applies Bcrypt/Argon2id
$options = ['cost' => 12]; // 2^12 iterations
$hashedPassword = password_hash($password, PASSWORD_BCRYPT, $options);
// Store $hashedPassword (60-char string) in MySQL VARCHAR(255)
```

2. Login Authentication (Verification):
```php
$stmt = $pdo->prepare('SELECT id, password_hash FROM users WHERE email = :email');
$stmt->execute(['email' => $email]);
$user = $stmt->fetch();

if ($user && password_verify($enteredPassword, $user['password_hash'])) {
// Password verified successfully! Mitigates timing attacks natively.
session_regenerate_id(true); // Prevent session fixation
$_SESSION['user_id'] = $user['id'];
} else {
// Generic error message to prevent user enumeration
$error = 'Invalid email or password.';
}
```

>>**Solution Statement Question 4:** Explain PHP Session Management mechanics, detailing session_start(), cookie parameters, session fixation, and secure session destr

3. Session Fixation Defense:
Whenever a user transitions from anonymous to authenticated, call `session_regenerate_id(true)`
to destroy the old session ID and issue a new one.
4. Complete Session Teardown (Logout):
```php
$_SESSION = [];
if (ini_get('session.use_cookies')) {
$params = session_get_cookie_params();
setcookie(session_name(), '', time() - 42000, $params['path'], $params['domain'],
$params['secure'], $params['httponly']);
}
session_destroy();
```

Cross-Site Scripting allows attackers to execute malicious JavaScript in victims' browsers, hijacking sessions or defacing pages:

Server-Side Defensive Architecture:
1. HTML Entity Encoding via `htmlspecialchars()`:
Before outputting any user-supplied variable into HTML context, convert special characters into safe HTML entities:
```php
echo '<p>Welcome, ' . htmlspecialchars($userInput, ENT_QUOTES | ENT_HTML5, 'UTF-8') . '</p>';
// Converts <script> into &lt;script&gt; rendering text visibly without execution
```

2. Context-Aware Escaping Rules:
- Inside HTML Body: Use `htmlspecialchars()`.
- Inside HTML Attributes (`<input value="...">`): Ensure attribute values are enclosed in quotes and encoded with `ENT_QUOTES`.
- Inside JavaScript Context (`<script>var data = ...;</script>`): Encode via `json_encode($data, JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT)`.
- Inside URLs: Use `urlencode()` and whitelist protocols (rejecting `javascript:` pseudoprotocols).

3. Defense-in-Depth: Enforce Content Security Policy (CSP) headers to block unauthorized script execution.

>>Solution to Question 57: Explain Cross-Site Request Forgery (CSRF) defense using Anti-CSRF Synchronizer Tokens and SameSite cookies.

```
$_POST['csrf_token'])) {
http_response_code(403);
die('CSRF token validation failed! Request rejected.');
}
// Process transfer safely
}
```

Using `hash_equals()` guarantees constant-time string comparison, neutralizing timing attack exploits.

>>Solution Question 8: Explain XML entity expansion attacks (Billion Laughs) and XML External Entity (XXE) injection. Provide secure configurations for PHP's XML parsers.

```
<body>
<h2>Product Catalog</h2>
<table border="1">
<tr bgcolor="#f2f2f2"><th>Title</th><th>Price</th></tr>
<xsl:for-each select="catalog/book">
<tr>
<td><xsl:value-of select="title"/></td>
<td>$<xsl:value-of select="price"/></td>
</tr>
</xsl:for-each>
</table>
</body>
</html>
</xsl:template>
</xsl:stylesheet>
```

>>Solution Question Bank RESTful API RESTful Web Services Stateless Client-Server Cacheable Uniform Interface and layered System.

3. Cacheability: Responses must explicitly define themselves as cacheable or non-cacheable (via `Cache-Control` headers) to prevent stale data while eliminating redundant server hits.
4. Uniform Interface: Standardized interaction contract across four tenets: identification of resources (URIs), manipulation through representations (JSON), self-descriptive messages, and HATEOAS.
5. Layered System: The client cannot tell whether it is connected directly to the end server or an intermediate proxy, CDN, or load balancer.
6. Code on Demand (Optional): Server temporarily extends client functionality by transferring executable code (e.g. JavaScript).

>>Solution Question 18 PSDN Web Token (JWT) detailed Files and validation workflow.

```
// 3. Validate true MIME type using finfo (never trust $_FILES['type']!)
$finfo = new finfo(FILEINFO_MIME_TYPE);
$mime = $finfo->file($file['tmp_name']);
$allowedMimes = ['image/jpeg' => 'jpg', 'image/png' => 'png', 'image/webp' => 'webp'];
if (!array_key_exists($mime, $allowedMimes)) die('Invalid file type.');

// 4. Generate random filename and store OUTSIDE web document root
$extension = $allowedMimes[$mime];
$safeFilename = bin2hex(random_bytes(16)) . '.' . $extension;
$destination = '/var/uploads/' . $safeFilename;

if (move_uploaded_file($file['tmp_name'], $destination)) {
echo 'File securely uploaded: ' . $safeFilename;
}
}
```

SSRF occurs when a web application accepts a user-controlled URL and fetches remote resources on behalf of the user without validation:

The Exploit Scenario:
Consider a feature that generates link previews: `fetch_url($_POST['image_url'])`.
- Attacker inputs: `http://169.254.169.254/latest/meta-data/iam/security-credentials/`
- The internal web server fetches the AWS cloud instance metadata service and returns secret IAM administrator credentials directly to the attacker!

Defensive Architecture:
1. Whitelisting Domains: Only allow fetching from pre-approved public domains.
2. Private IP Blacklisting: Resolve the domain IP via DNS and reject private/loopback CIDR ranges (127.0.0.0/8, 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, and cloud metadata 169.254.169.254).
3. Disable Following Redirects: Prevent attackers from redirecting from an approved public domain to internal IP addresses.

>>Solution Question Bank XML External Entity (XXE) Content Security Policy (CSP) Set to Transform Secure XML (HSTS) X-Content-Type-Options and X-Fra

1. Strict-Transport-Security (HSTS):
`Strict-Transport-Security: max-age=31536000; includeSubDomains; preload`
- Forces browsers to communicate exclusively over HTTPS for the next year, neutralizing SSL-stripping and downgrade attacks.

2. Content-Security-Policy (CSP):
`Content-Security-Policy: default-src 'self'; script-src 'self' https://trusted.cdn.com;`
- Restricts domains permitted to load and execute scripts, shutting down XSS payloads.

3. X-Content-Type-Options:
`X-Content-Type-Options: nosniff`
- Prevents browser from MIME-sniffing files away from the server's declared Content-Type, preventing image upload scripts from executing.

4. X-Frame-Options:
`X-Frame-Options: DENY` (or `SAMEORIGIN`)
- Completely blocks external sites from framing the page inside an `<iframe>`, preventing Clickjacking attacks.

>> Solution to Question 18 Part 2: Architecting Code Obfuscation and Proof of Work **Solution to Question 19** Part 1: Architecting Code Obfuscation and Proof of Work for Code Exchange Blockchain

empty, request is rejected with HTTP 429 Too Many Requests.

Redis Sliding Window Counter Implementation in PHP:

```php
$redis = new Redis(); $redis->connect('127.0.0.1');
$clientIp = $_SERVER['REMOTE_ADDR'];
$key = "rate:" . $clientIp;

$current = $redis->incr($key);
if ($current === 1) {
$redis->expire($key, 60); // Set 60-second window
}

if ($current > 100) { // Limit: 100 requests per minute
header('HTTP/1.1 429 Too Many Requests');
header('Retry-After: 60');
die(json_encode(['error' => 'Rate limit exceeded. Try again in 1 minute.']));
}
```

```
$id = isset($_GET['id']) ? (int)$_GET['id'] : null;

switch ($method) {
case 'GET':
if ($id) {
$stmt = $pdo->prepare('SELECT id, name, price FROM products WHERE id = ?');
$stmt->execute([$id]);
$data = $stmt->fetch();
echo json_encode($data ?: ['error' => 'Product not found']);
} else {
$stmt = $pdo->query('SELECT id, name, price FROM products LIMIT 50');
echo json_encode($stmt->fetchAll());
}
break;
case 'POST':
$body = json_decode(file_get_contents('php://input'), true);
$stmt = $pdo->prepare('INSERT INTO products (name, price) VALUES (?, ?)');
$stmt->execute([$body['name'], $body['price']]);
http_response_code(201);
echo json_encode(['id' => $pdo->lastInsertId(), 'message' => 'Created']);
break;
case 'PUT':
$body = json_decode(file_get_contents('php://input'), true);
$stmt = $pdo->prepare('UPDATE products SET name = ?, price = ? WHERE id = ?');
$stmt->execute([$body['name'], $body['price'], $id]);
echo json_encode(['message' => 'Updated']);
break;
case 'DELETE':
$stmt = $pdo->prepare('DELETE FROM products WHERE id = ?');
$stmt->execute([$id]);
echo json_encode(['message' => 'Deleted']);
break;
default:
http_response_code(405);
echo json_encode(['error' => 'Method Not Allowed']);
}
```

>> Solution to Question 22: Write a PHP class to generate and verify JSON Web Tokens (JWT) using HMAC-SHA256 without external third-party libraries.

```
$header = json_encode(['typ' => 'JWT', 'alg' => 'HS256']);
$base64Header = self::base64UrlEncode($header);
$base64Payload = self::base64UrlEncode(json_encode($payload));
$signature = hash_hmac('sha256', $base64Header . '.' . $base64Payload, self::$secret, true);
$base64Signature = self::base64UrlEncode($signature);
return $base64Header . '.' . $base64Payload . '.' . $base64Signature;
}

public static function verify($token) {
$parts = explode('.', $token);
if (count($parts) !== 3) return false;
list($header, $payload, $signature) = $parts;

$validSig = self::base64UrlEncode(hash_hmac('sha256', $header . '.' . $payload, self::$secret,
true));
if (!hash_equals($validSig, $signature)) return false;

$data = json_decode(base64_decode(strtr($payload, '-_', '+/')), true);
if (isset($data['exp']) && $data['exp'] < time()) return false; // Expired
return $data;
}
}
```

>>Solution to Question 23: Here is a complete XML Schema (XSD) document defining a validated Student Record with ID, name, email, and age constraints.

```
<xs:attribute name="id" type="xs:positiveInteger" use="required"/>
</xs:complexType>
</xs:element>
</xs:schema>
```

>>Solution Question 26. PHP how to prevent SQL injection in using DOMDocument with XPath queries in PHP PDO. over $50.

```
}
if (!empty($_GET['keyword'])) {
$conditions[] = 'title LIKE :keyword';
$params['keyword'] = '%' . $_GET['keyword'] . '%';
}

$sql = 'SELECT id, title, price FROM products';
if (count($conditions) > 0) {
$sql .= ' WHERE ' . implode(' AND ', $conditions);
}

$stmt = $pdo->prepare($sql);
$stmt->execute($params); // Safely passes only bound parameters
$products = $stmt->fetchAll();
```

>>Solution Question a PHP parser to transform raw XML documents into an HTML table. Use XSLT processors FastCGI and PHP-FPM architectures require c

```
// Output generated HTML table
echo $proc->transformToXML($xml);
```

>>Solution Question 28 Content Security Policy (CSP) Implementation in Web Applications Time-Based One-Time Password (TOTP) Algorithm

---

Solution to Question 30: Analyze Web Application security auditing tools: OWASP ZAP, Burp Suite, SQLMap, and Nikto in penetration testing workflows.